

Steps to set up a Redux store

1. Design store
2. Define actions
3. Define reducer 
4. Create store and make it available
5. Connect components to the store

Step 1 - Design store

- The whole global state of the application is stored inside a single store.
- Store can be any data type, usually a Javascript object

Step 2 – Define actions

- Action is an event that describes something that happened in the application.
- Actions are plain JavaScript objects that have
 - type - a string signifying which event happened
 - payload –any extra data needed to describe the action

Step 3 – Define reducer

- Reducers are functions that take the current state and an action as arguments and return a new state result.
- In other words, $(state, action) \Rightarrow newState$.
- State cannot be modified directly. Create a new state based on the old state.

Step 4 – Create the store and make it available

- **createStore(reducer)**
 - Creates a Redux store that holds the complete state tree of your app.
 - There should only be a single store in your app.

 BookTypes.jsx U X

my-app2 > src > Redux >  BookTypes.jsx >  buy_book

```
1  export const buy_book='buy_book'
```


BookReducer.jsx U ●

my-app2 > src > Redux > BookReducer.jsx > BookReducer

```
1  import React from 'react'
2  import { buy_book } from '../BookTypes'
3  const initialState = {
4    NumberOfBooks: 20
5  }
6  const BookReducer = (state = initialState, action) => {
7    switch (action.type) {
8      case buy_book: return {
9        ...state, NumberOfBooks: state.NumberOfBooks - 1
10      };
11      default: return state;
12    }
13  }
14  export default BookReducer;
```

BookContainer.jsx U

my-app2 > src > Redux > BookContainer.jsx > BookContainer

```
1  import React from 'react'
2  import { useDispatch, useSelector } from 'react-redux'
3  import purchase_book from './BookAction'
4  function BookContainer() {
5      const noofbooks= useSelector(state=>state.NumberOfBooks )
6      const callaction=useDispatch()
7      return (
8          <div>
9              <h1>No of Books {noofbooks}</h1>
10             <button onClick={()=>callaction(purchase_book())}>
11                 ⚠ buy Book </button>
12             </div>
13         )
14     }
15
16     export default BookContainer
```


 BookStore.jsx U X

my-app2 > src > Redux >  BookStore.jsx > [🔗] default

```
1  import { createStore } from "redux";
2  import { composeWithDevTools } from 'redux-devtools-extension';
3  import BookReducer from "../BookReducer";
4
5  const store=createStore(BookReducer, composeWithDevTools())
6
7  export default store;
```

Immutable Updates by Reducer

- Update a JSON
- Insert an item in an array
- Remove an item from an array
- Update an item in an array

Insert an item in array

- Slice the initial items in the array and spread them
- Add the new item
- Slice the remaining items in the array and spread them

```
[...array.slice(0,index),  
  item,  
  ...array.slice(index)  
]
```

Insert an item in array

- Slice the initial items in the array and spread them
- Add the new item
- Slice the remaining items in the array and spread them

```
[...array.slice(0,index),  
  item,  
  ...array.slice(index)  
]
```

Remove an item in array

- Filter the array
- Note : Filter creates a new array

```
array.filter(item => item.id !== action.id);
```



Update an item in array

- Use map.
- Update the item using the Update JSON approach

```
array.map((item) =>  
  (item.id !== action.id  
  ? item  
  : {...item,  
    xyz : action.xyz  
  })  
)
```


Exercise 1a : Store Design

```
{stocks:  
  [  
    { name: "Apple", quantity: 1 },  
    { name: "Banana", quantity: 2 },  
    { name: "Orange", quantity: 3 },  
  ]  
}
```

Action : SELL

- Updates quantity
- ```
{
 type : "SELL",
 payload : {
 name : fruit_name
 }
} // fruit_name is the name of the fruit
```

## Update a JSON

- Make a copy using the spread operator
- Then add the new fields and fields being updated

```
obj = {name: "Pepsi", qty:15, price:10}
```

Update : add discount with the value as 2

```
obj1 = {...obj, discount:2}
```